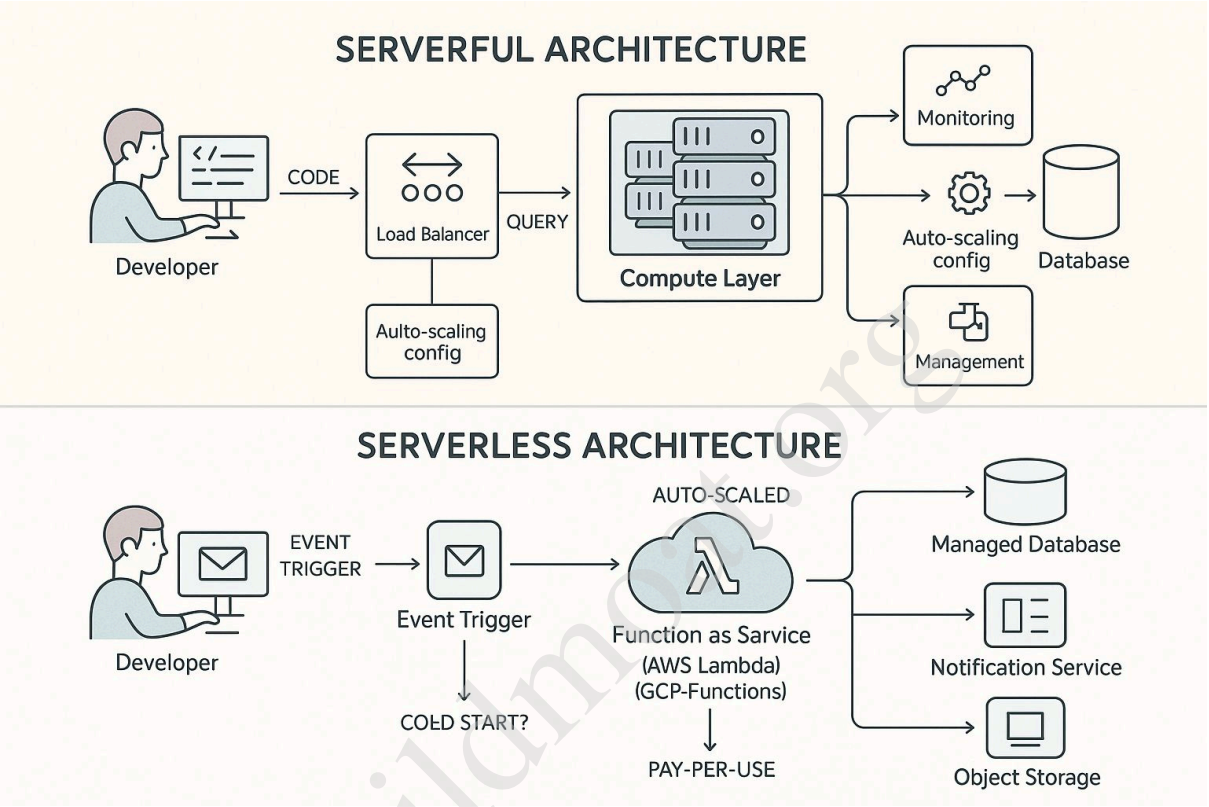
Serverless



什麼是 Serverless？

先澄清一件事：Serverless 不是「沒有伺服器」。伺服器當然還在，只是你不需要管它。

傳統的部署方式，你要租一台機器、安裝作業系統、設定執行環境、部署應用程式、監控它是否還活著，然後在流量增加時想辦法擴展。Serverless 把這些事全部交給雲端供應商，你只需要提供程式碼，告訴系統「當某件事發生時，執行這段程式碼」。

Serverless 有兩個主要的形態：

FaaS（Function-as-a-Service） 是最常被提到的那個。你的程式碼以「函式」為單位部署，由事件觸發執行——一個 HTTP 請求、一筆資料庫寫入、一個訊息進了 queue。函式執行完畢，資源立刻釋放。你只為函式真正執行的時間付費，計費精確到毫秒。代表產品是 AWS Lambda、Google Cloud Functions、Azure Functions。

buildmost.org

BaaS（Backend-as-a-Service） 是另一個面向。資料庫（Firebase Firestore、DynamoDB）、驗證服務（Auth0、Firebase Auth）、儲存（S3）——這些也是「無伺服器」的，因為你不管理任何伺服器，只是呼叫 API。在系統設計面試裡，提到 Serverless 通常是指 FaaS。

在系統設計的脈絡下，Serverless 的核心價值不只是「少管幾台機器」。它改變了你對工作負載的思考方式：從「這個服務需要幾台機器？」變成「這個事件觸發的計算量是多少、需要多快完成？」

Serverless 怎麼運作

理解底層機制，才能在面試中說清楚為什麼 Serverless 在某些場景下極為合適，在另一些場景下卻是糟糕的選擇。

事件驅動的執行模型

Serverless 函式不是一直在跑的 process，而是被事件喚醒、執行、然後消失。這些事件可以來自很多地方：

- **HTTP 請求**：API Gateway 接到一個 POST /orders，觸發 Lambda 函式處理訂單
- **訊息 Queue**：SQS 裡新增了一筆訊息，觸發函式處理
- **排程**：每天凌晨兩點觸發一次，跑批次報告
- **資料庫變更**：DynamoDB Stream 有新寫入，觸發下游處理
- **檔案上傳**：S3 有新圖片，觸發縮圖產生函式

這個模型讓你的系統在沒有流量的時候，真的什麼都不跑，也真的什麼都不花。對於流量不穩定、有明顯峰谷的工作負載，這是個很大的優勢。

buildroot.org

Cold Start 與 Warm Start

這是 Serverless 最重要的機制，也是最常在面試中被深挖的點。

第一次（或閒置太久後）呼叫一個函式時，雲端供應商需要：

1. 分配一個容器（或 microVM）來執行你的函式
2. 把你的程式碼載入進來
3. 初始化執行環境（Node.js runtime、Python interpreter 等）
4. 執行你函式裡的初始化程式碼（建立資料庫連線等）

這個過程叫做 **Cold Start**，通常需要幾百毫秒到幾秒不等，取決於執行環境的大小和初始化工作的複雜度。Java、.NET 這類有大量 JVM 或 CLR 初始化開銷的語言，cold

start 可能拉長到幾秒；Python 和 Node.js 通常幾百毫秒。

函式執行完後，雲端供應商不會立刻把容器銷毀，而是讓它「保溫」一段時間（通常 5 到 15 分鐘）。這段時間內如果有新請求進來，就直接用已經準備好的容器執行，不需要重新初始化——這叫 **Warm Start**，延遲和正常服務沒有差別。

```
Cold Start：
請求 → 分配容器 → 初始化環境 → 執行函式 → 回傳
        [──────── 額外延遲 300ms - 3s ────────]

Warm Start：
請求 → 執行函式 → 回傳
        [無額外延遲]
```

無狀態的執行單位

Serverless 函式是嚴格無狀態的。每次執行都是一個獨立的世界，函式不能假設上一次執行留下了什麼。本地的記憶體、臨時檔案，都可能在下次執行時消失。

這不只是一個限制，也是一個設計約束：任何需要跨請求保留的狀態，都必須外部化到資料庫、快取、或物件儲存裡。這和容器的無狀態設計原則是相同的，在 Serverless 環境下，這一點更為嚴格，因為函式執行環境是短暫且不可預測的，無法可靠地在記憶體中保留任何狀態或快取。

Serverless 與其他部署方式的比較

在面試中，清楚說出不同部署方式的取捨，比記住細節更重要。

	傳統伺服器 / VM	容器（Kubernetes）	Serverless（Lambda）
管理複雜度	高（OS、runtime、部署）	中（K8s 設定）	低（只管程式碼）
啟動速度	分鐘級	秒級	毫秒到秒（含 cold start）
擴展方式	手動或 Auto Scaling	HPA 自動擴展	完全自動、近乎無限
執行時間限制	無	無	15 分鐘（Lambda 上限）
閒置成本	高（永遠在付費）	中（需要底層節點）	零（沒執行就沒費用）
計費粒度	按小時 / 月	按 CPU/記憶體使用	按毫秒執行時間
延遲穩定性	穩定	穩定	Cold start 時不穩定

buildmost.org

	傳統伺服器 / VM	容器（Kubernetes）	Serverless（Lambda）
狀態管理	可本地狀態	通常無狀態	嚴格無狀態

沒有一個方案在所有場景下都最好。選擇的關鍵是工作負載的特性。

Serverless 適合什麼

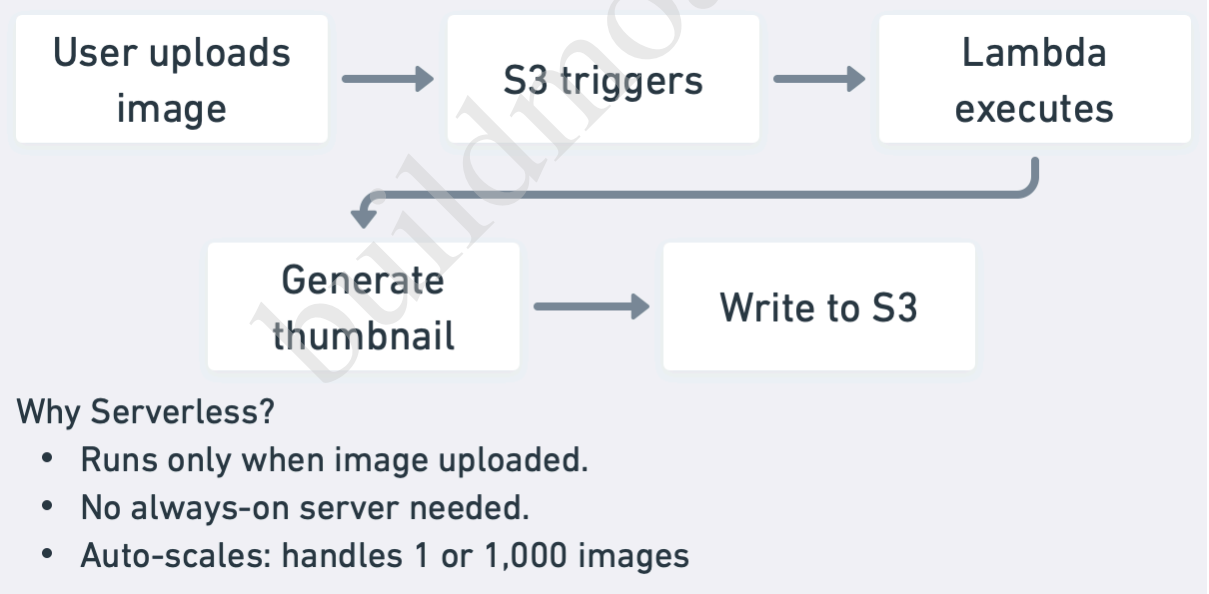
流量不穩定的工作負載

這是 Serverless 的主場。如果你的服務一天只有幾個小時有流量，其餘時間幾乎閒置，用傳統伺服器或容器，你要為那些閒置的資源付費。Serverless 讓你只為真正執行的那些毫秒付費。

新創公司早期的 API、內部工具、B2B 服務——這些場景下每天可能只有幾千個請求，Serverless 的成本遠低於維護一組始終運行的容器。

事件驅動的處理管線

Serverless 和訊息 queue、事件流的組合非常自然。



每個步驟都是獨立的函式，只在有事件時才執行。不需要維護一個永遠在輪詢 queue 的 worker process。

排程任務

每天凌晨跑一次的資料清理、每小時的報告匯總、每週的通知——這些工作如果用傳統方式，你要麼在應用程式裡管理 cron job，要麼維護一台專門跑排程任務的機器，它

buildnoat.org

99.9% 的時間都在閒著。

AWS EventBridge（前身是 CloudWatch Events）讓你用 cron 表達式觸發 Lambda 函式，費用通常是幾分錢一個月。

邊緣計算

AWS Lambda@Edge、Cloudflare Workers 讓函式在距離用戶最近的 CDN 節點執行，不需要請求跑回原始伺服器。這對地理位置敏感的操作（IP 地理定位、A/B 測試路由、請求認證）特別有用，因為延遲幾乎可以忽略不計。

Serverless 不適合什麼

知道什麼時候不用 Serverless，和知道什麼時候用一樣重要。

長時間執行的任務：Lambda 有 15 分鐘的硬性執行上限。影片轉碼、大型資料集處理、複雜的 ML 訓練——這些不適合 Serverless。

低延遲、延遲穩定性要求高的場景：如果你的 SLA 要求 P99 延遲低於 100ms，Cold start 帶來的不確定性是個問題。即時遊戲、高頻交易——Serverless 不適合。當然，有 Provisioned Concurrency 可以緩解，但那就讓你開始為閒置的容量付費，失去了 Serverless 的一大優勢。

持續高流量的服務：如果你的服務每秒有幾千個請求，從不間斷，Serverless 的按毫秒計費算下來可能比一組容器貴很多。大流量、穩定負載的場景，容器通常更划算。

需要複雜狀態管理的服務：WebSocket 長連線、需要本地快取、需要維護連線池的服務——Serverless 的無狀態特性讓這些很難做到高效。

buildroad.org

Serverless 架構的核心模式

API + Lambda 的組合

最常見的 Serverless 架構是 API Gateway + Lambda：

```
Client → API Gateway → Lambda 函式 → DynamoDB / RDS
```

API Gateway 負責接收 HTTP 請求、做認證（JWT 驗證、API key）、限流、然後把請求傳給對應的 Lambda 函式。每個路由可以對應到不同的函式，也可以讓一個函式處理所有路由（所謂的 Monolithic Lambda，在函式內部做路由）。

```
// 一個簡單的 Lambda 函式（Node.js）
exports.handler = async (event) => {
    // event.httpMethod, event.path, event.body 包含請求資訊
```

```
    const body = JSON.parse(event.body);

    // 處理邏輯
    const result = await processOrder(body);

    // 回傳符合 API Gateway 格式的回應
    return {
        statusCode: 200,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(result)
    };
};
```

處理資料庫連線

這是 Serverless 最惡名昭彰的痛點之一。

傳統的應用程式有一個連線池，和資料庫保持著固定數量的長連線。Lambda 函式是無狀態的，每個實例都需要建立自己的資料庫連線，而且 Lambda 可以瞬間擴展到幾千個並發實例——這會把資料庫的連線上限打爆。

PostgreSQL 預設支援幾百個連線，MySQL 也類似。幾千個並發 Lambda 函式同時嘗試連接，資料庫直接被打掛。

解法是 **RDS Proxy**（AWS）或 **PgBouncer**：一個連線池代理，讓 Lambda 函式連到代理，代理和資料庫之間維護一個小型的長連線池。Lambda 可以開幾千個實例，它們都連到代理，代理用少數幾十個連線服務它們。

```
Lambda 實例 × 1000 → RDS Proxy（連線池）→ RDS（50 個連線）
```

DynamoDB 是另一個選擇。它是原生的 Serverless 資料庫，沒有連線的概念，按請求計費，天然適合 Lambda。如果你的資料模式允許，DynamoDB + Lambda 是一個很自然的全 Serverless 組合。

Fan-out 模式

一個事件觸發多個平行的處理流程：

```
新訂單 → SNS Topic → Lambda A（更新庫存）
                   → Lambda B（發確認信）
                   → Lambda C（通知物流）
```

buildmoat.org

SNS（Simple Notification Service）把一個訊息 fan-out 給多個 Lambda 函式，它們平行執行，互相獨立。某個函式失敗，不影響其他函式。這和 Pub/Sub 模式本質上相同，Serverless 讓實作這個模式變得非常輕量。

什麼時候在面試裡用這些

主動提出 Serverless 的時機

當工作負載有明顯的事件觸發性質

「用戶上傳圖片後需要產生縮圖。這個操作完全是事件驅動的——有圖片上傳就觸發，沒有就不跑。我會用 S3 Event Notification 觸發 Lambda 函式，函式讀取原始圖片、產生多種尺寸、寫回 S3。不需要維護任何常駐的 worker 服務，成本幾乎是零，除非有圖片進來。」

當流量模式高度不穩定

「這是個內部工具，白天有幾百個工程師在用，晚上幾乎沒有流量。用 Lambda + API Gateway，白天自動擴展到幾百個並發，晚上完全不跑、不花錢。比維護一組永遠在跑的容器便宜很多。」

當討論排程任務

「這個每天跑一次的資料清理任務，如果用傳統 cron job，要麼把它塞進應用程式、要麼維護一台專門的機器。我傾向用 EventBridge 排程觸發 Lambda，不需要任何基礎設施，費用微乎其微。」

說清楚何時不用 Serverless

面試官欣賞你能主動說「這個場景不適合 Serverless」：

「我們的支付 API 每秒處理幾千筆交易，負載非常穩定。這種情況下 Serverless 的彈性沒有什麼用，反而因為計費模型，成本會比容器高。我們用 Kubernetes 部署固定規模的服務，配合 HPA 處理少量的流量波動。」

「這個音訊轉碼任務需要跑 20 分鐘。Lambda 有 15 分鐘的硬性上限，所以這個不能用 Serverless。我們用一個跑在 EC2 Spot Instance 上的 worker，接 SQS 的 queue。」

常見面試情境

電商訂單通知系統：「訂單成立後需要發確認信、通知倉庫、更新積分。我用 SNS fan-out 到三個 Lambda 函式，平行處理。失敗的函式有各自的 Dead Letter Queue，不影響其他通知管道。整個系統只在有新訂單時才跑，成本和訂單量完全正相關。」

buildnoat.org

圖片處理平台：「用戶上傳圖片後，需要產生縮圖、做 NSFW 檢測、提取 EXIF metadata。S3 Upload 事件觸發一個 Lambda 函式，這個函式把三個處理任務分別丟進不同的 SQS queue，由三個不同的 Lambda 函式平行處理。縮圖產生完立刻可用；NSFW 檢測結果更新資料庫的審核狀態；EXIF 資料寫進搜尋索引。」

定期清理過期資料：「我們有個策略，用戶刪帳號後資料要在 30 天內完全清除。我不在用戶請求的 API 裡做這件事——那太慢了。每天用 EventBridge 觸發一個 Lambda，掃描所有刪除超過 30 天的帳號，批次清理。Lambda 的 15 分鐘限制對每天幾百個帳號完全夠用。如果量大到單次跑不完，就讓 Lambda 把任務分批，自己觸發自己（recursive invocation）或者丟到 SQS 繼續。」

常見的 Deep Dive 問題

「Cold Start 怎麼辦？」

這是 Serverless 的標誌性問題，面試官幾乎必問。

首先，承認問題存在，然後說清楚它在什麼情況下是問題、在什麼情況下不是：

Cold Start 對延遲穩定性要求高的場景（同步 API、P99 SLA < 100ms）是個真實問題。有幾個緩解策略：

減少 Cold Start 時間本身：選用 Cold Start 較快的語言（Python、Node.js 比 Java 快很多）；讓函式的部署包越小越好，去掉不需要的依賴；把初始化程式碼（資料庫連線、設定載入）放在函式 handler 外面，讓 Warm Start 可以重用：

```
# 把這段放在 handler 外面，只在 Cold Start 時執行一次
db = create_connection()  # warm start 時不會重複執行

def handler(event, context):
    # 直接用已有的 db 連線
    result = db.query(...)
    return result
```

Provisioned Concurrency：預先保留一定數量的「熱」實例，完全消除這些實例的 Cold Start。代價是你要為這些預留的實例付費，即使它們沒在處理請求。適合有流量預測能力的場景——白天預留 50 個實例，晚上縮回到 5 個。

Warm-up 機制：用排程定期觸發函式，讓它保持「熱」的狀態。這是個土方法，但在成本敏感的場景下有效。

非同步場景完全不需要擔心：如果你的 Lambda 是非同步處理（SQS 觸發、排程執行），Cold Start 多花幾百毫秒完全無所謂。只有同步 API 請求才需要認真考慮這個問

buildmoat.org

題。

「Serverless 的成本怎麼估算？」

面試中展示你能做粗略的成本估算，是加分點。

Lambda 的計費有兩個維度：**請求次數**（每百萬次請求約 0.2 美元）和**執行時間 × 記憶體**（每 GB-秒約 0.000016 美元）。

舉個例子：假設一個服務每天 100 萬個請求，平均每次執行 200ms，使用 512MB 記憶體：

```
每月請求費用：30 × 1M 請求 × $0.2/1M = $6
每月執行費用：30 × 1M × 0.2s × 0.5GB × $0.000016/GB-s = $48
合計：約 $54/月
```

對比一下 EC2 t3.small（2GB，2 vCPU）：約 $15/月，不限請求量。所以如果流量夠大（持續穩定），傳統機器可能更便宜。但如果流量在一天中有 12 小時接近零，Serverless 的實際費用可能是估算的一半甚至更少。

在面試中，你不需要算出精確數字。展示你知道兩個計費維度、知道在什麼流量曲線下 Serverless 更划算，就足夠了。

「怎麼處理 Serverless 的分散式追蹤和除錯？」

Serverless 系統的可觀察性是個真實的痛點。函式執行是短暫的，出了問題你沒辦法 SSH 進去查看。

結構化日誌：每個函式的 log 要帶有 correlation ID（可以從請求 header 傳入），讓你能在大量分散的 log 裡追蹤一個請求的完整鏈路：

```
import json, logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

def handler(event, context):
    correlation_id = event.get('headers', {}).get('X-Correl
ation-Id', context.aws_request_id)
    logger.info(json.dumps({
        "correlation_id": correlation_id,
        "action": "order_processing_started",
```

buildnoat.org

```
        "order_id": event['body']['order_id']
    }))
```

分散式追蹤：AWS X-Ray、Datadog APM 可以把跨多個 Lambda 函式、跨 SQS、跨 DynamoDB 的請求鏈路串起來，讓你看到一個請求從入口到結束的完整時序圖。在 Serverless 架構裡，這是幾乎必備的工具。

Dead Letter Queue 監控：DLQ 是你的早期預警系統。如果函式執行失敗，訊息進了 DLQ，CloudWatch 對 DLQ 深度設告警，讓你在問題擴大之前發現。

「Serverless 有 vendor lock-in 的問題嗎？」

這是個很好的問題，展示你在思考長期架構影響。

是的，Serverless 的 vendor lock-in 比容器明顯得多。Lambda 函式用的事件格式（API Gateway event、S3 event）是 AWS 特定的；DynamoDB 的 API 是 AWS 特定的；EventBridge 的規則語法是 AWS 特定的。如果有一天你想從 AWS 搬到 GCP，幾乎要重寫所有的整合程式碼。

緩解的方式：

在函式的 handler 裡加一層抽象，把雲端特定的邏輯和業務邏輯分開：

```
def handler(event, context):
    # 只有這一層是 AWS 特定的
    order = parse_api_gateway_event(event)
    # 這層是純業務邏輯，不依賴任何雲端 SDK
    result = process_order(order)
    return format_api_gateway_response(result)
```

業務邏輯可以在本地測試，也可以理論上移植到其他平台。但說實話，這只是緩解，不是根治——如果你深度使用了 SQS、DynamoDB、EventBridge 的整合，搬遷的成本還是很高的。

在面試中直接說：「Serverless 有明顯的 vendor lock-in，我們接受這個取捨，換取不需要管理基礎設施、自動擴展、以及降低運維複雜度的好處。如果未來業務規模大到需要考慮跨雲，我們可以在那個時候把高流量的服務遷移到容器，Serverless 的部分保留在流量不穩定的邊緣使用案例。」

總結

Serverless 的本質是一個**取捨**，而不是一個萬用解。它在一些場景下極其強大：事件驅動的處理管線、流量不穩定的 API、排程任務、邊緣計算——這些地方 Serverless 讓

bulanoat.org

你用很低的成本和維運負擔，獲得幾乎無限的自動擴展能力。

但它有幾個硬限制：15 分鐘的執行時間上限、Cold Start 帶來的延遲不穩定性、嚴格的無狀態設計要求、以及顯著的 vendor lock-in。這些不是可以繞過的小問題，而是在選擇 Serverless 之前就要接受的設計約束。

在系統設計面試中，提到 Serverless 的最佳時機是當工作負載明顯符合事件驅動模式、或流量曲線有大量閒置時間。說清楚你選擇 Serverless 的理由（事件觸發、流量不穩定、不想管基礎設施），以及你知道它的限制在哪裡（執行時間、Cold Start、連線管理）。

最有說服力的回答不是「我用 Lambda」，而是「我在這個部分用 Lambda，因為它是事件觸發的、流量峰谷明顯；但這個需要長時間執行的部分用 EC2 Worker，因為 Lambda 的 15 分鐘上限不夠用」。展示你在為每個工作負載選擇最合適的工具，而不是把 Serverless 當成一個時髦詞到處套用。

buildmoat.org